

STRING	CHARACTER
s.length() -> size (WITH ())	Character.isDigit(c) 0-9 ?
s.charAt(i) -> char at i	Character.isLetter(c) a-z/A-Z ?
s.substring(1,4) -> [1,4) end EXCL	Character.isLetterOrDigit(c)
s.indexOf('l') -> first index	Character.isUpperCase(c)
s.contains("el") -> true/false	Character.isLowerCase(c)
s.toCharArray() -> char[]	Character.isWhitespace(c)
s.toLowerCase()/toUpperCase()	Character.toLowerCase/toUpperCase(c)
s.equals(x) / s.equalsIgnoreCase(x)	
s.replace('l','x')	
s.split(" ") -> String[]	TRICK: char <-> index int idx = c - 'a'; // a=0,b=1 char ch = (char)('a' + 3); // 'd'
s.trim() -> remove spaces	
s.startsWith/endsWith(x)	
s.isEmpty()	
Integer.parseInt("123") -> int	- STRINGBUILDER (mutable, FAST) -
String.valueOf(123) -> String	sb.append(x) sb.insert(0,x)
	sb.reverse() sb.deleteCharAt(i)
	sb.charAt(i) sb.length()
	sb.toString()
	REVERSE 1-liner: new StringBuilder(s).reverse() .toString();

ARRAYS (import java.util.Arrays;)
Arrays.sort(a) -> ascending a.length (NO () !)
Arrays.toString(a) -> "[1, 2, 3]" 2D: grid.length = rows
Arrays.equals(a,b) -> same arrays? grid[0].length = cols
Arrays.fill(a,0) -> fill all
Arrays.copyOf(a,2) / copyOfRange(a,1,3) DESC sort (Integer[] only):
Arrays.asList(1,2,3) -> array to List Arrays.sort(arr, Collections.reverseOrder())

MAP (HashMap - MOST USED)	SET (HashSet - duplicates)
map.put(k,v)	set.add(x) // false if exists
map.get(k) -> null if none	set.contains(x)
map.getOrDefault(k,0) -> safe get	set.remove(x)
map.containsKey(k) / containsValue(v)	set.size() set.isEmpty()
map.remove(k) map.size() isEmpty()	set.addAll(list)
map.keySet() / values() / entrySet()	
map.equals(map2)	GOLDEN duplicate trick: if(!set.add(x)){ /*DUPLICATE!*/ }
*** FREQUENCY (memorize!) ***	TreeSet=sorted LinkedHashSet=insert
map.put(c, map.getOrDefault(c,0)+1);	
map.merge(c, 1, Integer::sum); //alt	
ITERATE:	- STACK & QUEUE -
for(Map.Entry<K,V> e:map.entrySet()){	STACK (LIFO): Deque<Integer> st = new ArrayDeque<>();
e.getKey(); e.getValue();	st.push(x) st.pop() st.peek()
}	QUEUE (FIFO): Queue<Integer> q = new LinkedList<>();
	q.offer(x) q.poll() q.peek()

LIST (ArrayList)	MATH
list.add(x) list.add(0,x)	Math.max(a,b) Math.min(a,b)
list.get(i) list.set(i,x)	Math.abs(-5)=5
list.remove(0) // INDEX !	Math.pow(2,3)=8.0 (double! cast it)
list.remove(Integer.valueOf(5))//VALUE	Math.sqrt(16)=4.0
list.contains(x) list.indexOf(x)	Math.floor(3.7)=3.0 ceil(3.2)=4.0
list.size() // NOT length!	Math.round(3.5)=4
Collections.sort(list) / reverse(list)	(int)Math.pow(2,3)=8
Collections.max(list) / min(list)	

JAVA 8 STREAMS (experienced = MUST)
nums.stream().filter(n->n>3).collect(Collectors.toList()); // keep matches
nums.stream().map(n->n*2).collect(Collectors.toList()); // transform
nums.stream().distinct() / .sorted() / .count()
nums.stream().mapToInt(Integer::intValue).sum(); // sum
nums.stream().max(Integer::compare).get(); // max
nums.stream().collect(Collectors.groupingBy(n->n,Collectors.counting()));//freq
nums.stream().distinct().sorted(Comparator.reverseOrder())
.skip(1).findFirst().get(); // 2nd highest
INTERMEDIATE(lazy): filter map sorted distinct limit TERMINAL: collect count sum forEach

```
TOP 5 PATTERNS (solve 50%+ of all questions)
```

[1] FREQUENCY COUNTER (HashMap)	[2] TWO POINTER
Map<Character,Integer> m = new HashMap<>(); for(char c: s.toCharArray()) m.put(c,m.getOrDefault(c,0)+1);	int l=0, r=arr.length-1; while(l<r){ // do work l++; r--; }
[3] FREQ ARRAY (lowercase only)	[4] SET FOR DUPLICATES
int[] f = new int[26]; for(char c: s.toCharArray()) f[c-'a']++;	Set<Integer> seen=new HashSet<>(); for(int n: arr) if(!seen.add(n)){/*dup*/}
[5] SLIDING WINDOW (substr/subarray)	
int start=0; for(int end=0; end<arr.length; end++){ // expand with arr[end] while(/*invalid*/){ start++; } // shrink }	

PROBLEM -> FORMULA MAP (quick lookup)

- Count frequency HashMap + getOrDefault / merge
- Find duplicates HashSet + !set.add(x)
- Check anagram int[26], c-'a' indexing (or sort both)
- Reverse string StringBuilder.reverse()
- Palindrome two-pointer (l++, r--)
- First non-repeat char LinkedHashMap + count==1
- Sort Arrays.sort / Collections.sort
- 2nd highest / Nth sort + skip OR stream distinct.skip(1)
- Remove dups from list new HashSet<>(list)
- Group by something Collectors.groupingBy
- Brackets / undo / reverse Deque push / pop / peek
- Compare two arrays Arrays.equals(a,b)
- char is digit/letter? Character.isDigit / isLetter
- Max / Min in array Collections.max/min OR loop

TIME COMPLEXITY (Big-O) – say this in interview!

- O(1) constant map.get(), array[i]
- O(log n) binary search
- O(n) single loop (most efficient solutions)
- O(n log n) sorting Arrays.sort()
- O(n²) nested loop (brute force – AVOID if possible)
- TIP: HashMap/HashSet => O(n) "efficient" | nested loop => O(n²) "brute"

SOLVE STRATEGY (do this for EVERY problem)

Step 1: Understand what's asked	> null input? (s==null)
Step 2: Identify pattern (freq? 2-ptr? set?)	> empty? (length==0)
Step 3: Pick formula (Sheet 1)	> single element?
Step 4: THEN write code	> duplicates allowed?
RULE: Speak your LOGIC first, code later.	> negative numbers?
Interviewer wants your THINKING.	> very large input?

COMMON TRAPS (don't lose marks here!)

s.length() -> WITH () (String)	substring(1,4) -> index 4 EXCLUDED
a.length -> NO () (Array)	list.remove(2) -> removes INDEX 2
list.size() -> size, not length (List)	Math.pow() -> returns double, cast!
== compares references; use .equals() for objects	
HashMap has NO guaranteed order (use LinkedHashMap for insertion order)	

10 JAVA 8 STREAMS (one-liners – experienced ki must)

```
```java
List<Integer> nums = Arrays.asList(5,2,8,1,2);

// filter -> condition pass ayyevi
nums.stream().filter(n -> n > 3).collect(Collectors.toList()); // [5,8]

// map -> transform
nums.stream().map(n -> n * 2).collect(Collectors.toList());

// distinct -> duplicates teesi
nums.stream().distinct().collect(Collectors.toList()); // [5,2,8,1]

// sorted
nums.stream().sorted().collect(Collectors.toList());

// count
nums.stream().filter(n -> n > 3).count(); // 2

// sum / max / min / average
nums.stream().mapToInt(Integer::intValue).sum(); // 18
nums.stream().max(Integer::compare).get(); // 8

// grouping (count frequency!)
nums.stream().collect(Collectors.groupingBy(n -> n, Collectors.counting()));
// {1=1, 2=2, 5=1, 8=1}

// 2nd highest
nums.stream().distinct().sorted(Comparator.reverseOrder())
 .skip(1).findFirst().get(); // 5
```

"Hi, I'm Sai Kiran. I have over four years of experience as a Java Full Stack Developer.

Currently, I work at Cognizant, developing mission-critical applications for American Airlines. Most recently, I engineered a real-time eventing framework using Apache Flink and Kafka that processes over 100,000 flight disruption events daily with sub-second latency. My core backend stack includes Java 11 and 17, Spring Boot, and Azure Kubernetes Service (AKS), while on the frontend, I build responsive UIs using Angular. Throughout my projects, I've focused heavily on system reliability—implementing things like exactly-once processing semantics and Dead Letter Topics to ensure zero message loss.

I'm here today because I'm looking to bring my expertise in scalable architecture to a dynamic and challenging environment."